

Université de Ngaoundéré

Master 2

Data Mining, Fouille de données
et Statistiques décisionnelles

Chapitre

2

Classification supervisée et évaluation des modèles

Support de cours

Dispensé par

Pr Ndam Njoya Arouna

Chapitre 2

Classification supervisée et évaluation des modèles

Table des matières

1	Introduction	2
1.1	Quelques définitions	2
2	Cadre statistique de la classification	3
2.1	Données et objectif	3
2.2	Trop simple ou trop compliqué?	3
3	Méthode des k plus proches voisins	4
4	Régression logistique	4
5	Classifieur Naïve Bayes	4
6	Arbres de décision	5
6.1	Règles IF-THEN extraites d'un arbre	5
7	Matrice de confusion et métriques	5
8	Courbe ROC, AUC et choix de métrique	6
9	Validation croisée et pipelines	7
10	Classes déséquilibrées	8
11	Courbe précision-rappel	9
12	Ensembles : bagging et boosting	9
13	Régularisation et sur-apprentissage en logistique	9
14	Questions de révision guidées	9
15	Erreurs fréquentes en évaluation	10
16	Synthèse méthodologique	10
17	Lecture des odds et rapport d'évaluation	10

1 Introduction

Le chapitre 1 a préparé le tableau. Ici, on apprend à **prédire une étiquette** connue sur les anciennes lignes (spam oui/non, malade oui/non, etc.) pour l'anticiper sur de **nouvelles** lignes. C'est l'**apprentissage supervisé**. On connaît la réponse y sur les données d'apprentissage et un **classifieur** (formule ou modèle) apprend à prédire y pour une ligne nouvelle. L'objectif n'est pas seulement d'afficher un pourcentage de réussite. Il faut comprendre **quelles erreurs** coûtent cher (fausse alerte vs alerte manquée), comparer plusieurs méthodes de façon équitable, et obtenir des **probabilités** utiles au chapitre 4.

Objectifs pédagogiques

- expliquer l'idée de sur-apprentissage (modèle trop collé aux données d'entraînement) ;
- utiliser k-NN, régression logistique, Naïve Bayes et un arbre de décision ;
- lire une matrice de confusion, précision, rappel, courbe ROC ;
- évaluer un modèle par validation croisée avec la chaîne de prétraitement ;
- rédiger un court compte rendu clair.

1.1 Quelques définitions

Variables explicatives et cible. Chaque ligne est un individu i décrit par des **variables explicatives** $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$ et par une **cible** y_i que l'on cherche à prédire. Le modèle renvoie une prédiction $\hat{y}_i$. En **classification binaire**, $y_i \in \{0, 1\}$ (deux classes seulement). *Exemple.* Pour un crédit, $\mathbf{x}_i$ peut contenir le revenu et le ratio d'endettement ; $y_i = 1$ si défaut, 0 sinon. Pour un filtre anti-spam, $y_i = 1$ si le message est un spam.

Apprentissage supervisé. Situation où, sur les données d'apprentissage, la vraie valeur de y est connue pour chaque ligne. On estime une règle à partir de ces exemples, puis on l'applique à des lignes nouvelles dont y est inconnu. *Exemple.* À partir de milliers de mails déjà étiquetés spam ou non, on entraîne un classifieur pour trier les prochains messages entrants.

Classifieur. Modèle ou algorithme qui calcule $\hat{y} = f(\mathbf{x})$ (k-NN, arbre, régression logistique, etc.). Il produit une classe ou une probabilité ; ce n'est pas encore l'action métier (refuser un prêt, bloquer un mail). *Exemple.* Une régression logistique peut renvoyer une probabilité de défaut de 0,73 ; prédire « défaut » si probabilité $> 0,5$ est une convention de seuil, pas la décision finale (chapitre 4).

Jeu d'entraînement. Partie des données réservée à l'apprentissage : c'est sur ce jeu que l'on appelle **fit** pour estimer les paramètres du modèle (coefficients, structure de l'arbre, etc.). *Exemple.* Sur 1000 dossiers crédit, les 700 premières lignes (ou 70 % tirés au hasard) peuvent constituer le train.

Jeu de validation. Partie utilisée pour comparer plusieurs réglages ou modèles candidats, sans toucher au jeu de test final. *Exemple.* On teste $k = 3$ et $k = 15$ en k-NN sur la validation et on retient le k qui donne la meilleure AUC sur cette partie seulement.

Jeu de test. Partie gardée jusqu'à la fin du projet pour mesurer une fois la performance sur des données non utilisées à l'entraînement ni au réglage. *Exemple.* Les 150 derniers dossiers crédit (15 %) servent de test après avoir fixé le modèle sur train et validation.

Pipeline. Objet scikit-learn qui enchaîne prétraitement et modèle en une seule étape entraînée ensemble (`fit` sur le train, `transform` ou `predict` sur le test). *Exemple.* `Pipeline([StandardScaler(), LogisticRegression()])` : le scaler apprend moyenne et écart-type sur le train uniquement, puis le même scaler transforme le test.

Sur-apprentissage. Phénomène où le modèle s'ajuste trop finement aux données d'entraînement (y compris le bruit) et généralise mal sur des données nouvelles. *Exemple.* Un arbre très profond ou k-NN avec $k = 1$ atteint parfois une exactitude parfaite sur le train mais chute fortement sur le test.

Sous-apprentissage. Phénomène où le modèle est trop simple pour représenter la relation entre $\mathbf{x}$ et y ; les performances restent faibles sur le train comme sur le test. *Exemple.* Un arbre avec une seule question sur le revenu peut mal prédire le défaut même sur les dossiers déjà vus à l'entraînement.

2 Cadre statistique de la classification

2.1 Données et objectif

On dispose de n exemples. Pour chaque ligne, les variables $\mathbf{x}_i$ et la réponse y_i . On cherche une règle f qui commet le moins d'erreurs possible sur des données **nouvelles**. En binaire, une erreur = prédire 0 alors que la vérité est 1, ou l'inverse. Le risque théorique s'écrit :

$$R(f) = \mathbb{E}_{(\mathbf{x}, y)} [\mathbb{1}_{f(\mathbf{x}) \neq y}] \quad (1)$$

Les logiciels ajustent la règle sur les données vues ; si la règle est trop complexe, elle **mémorise** le bruit. C'est le **sur-apprentissage**. Le **classifieur** apprend une règle f à partir d'exemples où la réponse y est connue ; l'enjeu est la **généralisation** sur des données nouvelles. Le **sous-apprentissage** correspond au cas inverse : un modèle trop simple ne capture pas la structure utile.

2.2 Trop simple ou trop compliqué ?

L'objectif n'est pas d'avoir le meilleur score sur les lignes déjà vues à l'entraînement, mais sur des **nouvelles** lignes. Deux erreurs fréquentes vont dans des directions opposées.

Sous-apprentissage. Le modèle est trop rigide ou trop pauvre pour décrire le lien entre $\mathbf{x}$ et y . Il se trompe souvent *déjà* sur le train, et autant sur le test : il n'a pas « appris » assez. *Exemple.* Un arbre limité à une seule question (« endettement > 50 % ? ») peut laisser passer des défauts chez les clients à faible ratio mais à revenu insuffisant. Les métriques train et test restent toutes deux médiocres. Pistes : ajouter des variables, assouplir le modèle (arbre un peu plus profond, frontière non linéaire via k-NN ou logistique avec interactions).

Sur-apprentissage. Le modèle est trop flexible : il s'adapte aux moindres détails du fichier d'entraînement, y compris au hasard (bruit de mesure, dossiers atypiques). Il affiche alors un excellent score sur le train, mais se dégrade sur le test. *Exemple.* k-NN avec $k = 1$ attribue à une nouvelle ligne la classe de son unique voisin le plus proche ; une petite erreur de saisie sur une variable peut faire basculer la prédiction. Un arbre sans limite de profondeur peut créer une feuille

par dossier et mémoriser chaque client. Pistes : augmenter k , limiter la profondeur de l'arbre, régulariser la logistique (section dédiée plus loin).

Compromis et validation. Entre ces deux extrêmes, on cherche un réglage qui généralise bien. On compare plusieurs candidats (valeurs de k , profondeur max, force de régularisation) sur un jeu de **validation** ou par validation croisée sur le train, puis on évalue une dernière fois sur le **test** gelé. Changer le test à chaque essai pour « trouver le meilleur chiffre » fausserait la conclusion : le test ne sert qu'à confirmer le choix déjà fait sur train/validation.

3 Méthode des k plus proches voisins

k plus proches voisins (k-NN) est une méthode dite **lazy**. On conserve tout le tableau et, pour une nouvelle ligne, on vote la classe majoritaire parmi les k voisins les plus proches (distance euclidienne; variables standardisées au chapitre 1). k petit donne des décisions très locales, instables; k grand donne des décisions plus stables mais parfois trop lisses.

4 Régression logistique

La régression logistique estime une **probabilité** entre 0 et 1 :

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(u) = \frac{1}{1 + e^{-u}} \quad (2)$$

Les paramètres $\mathbf{w}$ sont choisis pour que les probabilités prédites soient les plus cohérentes possibles avec les y_i observés (minimisation de l'entropie croisée) :

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (3)$$

Chaque coefficient indique dans quel sens une variable pousse la probabilité. Par défaut, on prédit « oui » si $\hat{p} \geq 0,5$. Au chapitre 4, on changera ce seuil selon le coût des erreurs. La régression logistique est un outil de **classification binaire**. Elle estime une probabilité via la sigmoïde, et l'entropie croisée sert de fonction de coût pour ajuster les coefficients $\mathbf{w}$.

5 Classifieur Naïve Bayes

On combine une probabilité a priori de classe $P(y = c)$ et la vraisemblance des données. Le théorème de **Bayes** :

$$P(y = c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y = c) P(y = c)}{P(\mathbf{x})} \quad (4)$$

Naïve Bayes suppose que les colonnes sont **indépendantes entre elles** une fois la classe connue (hypothèse forte, souvent fausse, mais pratique) :

$$P(\mathbf{x} \mid y = c) = \prod_{j=1}^p P(x_j \mid y = c) \quad (5)$$

La règle de Bayes devient :

$$\hat{y} = \arg \max_c P(y = c) \prod_{j=1}^p P(x_j | y = c) \quad (6)$$

Les variantes gaussiennes (variables continues) ou multinomiales (texte) sont rapides et robustes en haute dimension. $P(y = c)$ est la fréquence a priori de la classe; $P(\mathbf{x} | y = c)$ mesure la compatibilité des variables. L'hypothèse « naïve » d'indépendance conditionnelle est souvent fausse, mais le classifieur reste une baseline utile.

6 Arbres de décision

Un arbre partitionne récursivement l'espace des features en régions homogènes pour y . À chaque nœud, on choisit le split maximisant un critère (**entropie** ou **Gini**). Entropie de Y au nœud t :

$$H(Y | t) = - \sum_{c \in \mathcal{Y}} p(c | t) \log_2 p(c | t) \quad (7)$$

Gain d'information pour un attribut A :

$$\text{IG}(A) = H(Y | t) - \sum_v \frac{n_v}{n} H(Y | t_v) \quad (8)$$

Indice de Gini : $G = 1 - \sum_c p(c | t)^2$. On peut aussi utiliser le **gain de ratio de Gini** et l'**élagage**, qui consiste à couper les branches qui n'améliorent plus la performance sur un jeu de validation. Sans contrainte, l'arbre mémorise le bruit. Les arbres se lisent comme des règles « si ... alors ... », utiles en réunion métier.

6.1 Règles IF-THEN extraites d'un arbre

On peut extraire des règles lisibles depuis un arbre ou les construire par **couverture séquentielle**. On cherche d'abord une règle qui couvre beaucoup de positifs, retirer ces lignes, recommencer. Exemple : « si revenu < seuil et endettement > seuil alors défaut ». Moins précis qu'un arbre complet, parfois plus facile à valider juridiquement.

7 Matrice de confusion et métriques

Pour la classe positive 1 (ex. défaut, fraude) :

	Prédit 1	Prédit 0
Réel 1	TP (vrai alerte)	FN (alerte manquée)
Réel 0	FP (fausse alerte)	TN

$$\text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel (sensibilité)} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (9)$$

La matrice de confusion croise **réalité** et **prédiction**. **TP** et **TN** sont les bonnes décisions, **FP** et **FN** les erreurs (la **classe positive** est celle qu'on cherche à repérer, par exemple le défaut). L'**exactitude** $(TP + TN)/n$ masque la performance sur la classe rare. Avec 99 % de bons payeurs,

		Predicted class			
			<i>yes</i>	<i>no</i>	Total
Actual class	<i>yes</i>		<i>TP</i>	<i>FN</i>	<i>P</i>
	<i>no</i>		<i>FP</i>	<i>TN</i>	<i>N</i>
	Total		<i>P'</i>	<i>N'</i>	<i>P + N</i>

FIGURE 1 – Matrice de confusion avec totaux par classe prédite et réelle. (Source : Han, Kamber et Pei, *Data Mining: Concepts and Techniques*, 3e éd., Figure 8.14.)

un modèle prédisant toujours « bon » atteint 99% d'exactitude sans pouvoir discriminant. La **spécificité** $TN/(TN + FP)$ mesure les vrais négatifs bien détectés. En déséquilibre, privilégier rappel, F_1 ou courbe précision-rappel.

8 Courbe ROC, AUC et choix de métrique

En faisant varier le seuil τ sur $\hat{p}$:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \quad \text{FPR}(\tau) = \frac{FP}{FP + TN} \quad (10)$$

La courbe ROC trace le taux de vrais positifs (TPR, rappel) en fonction du taux de faux positifs (FPR) lorsque le **seuil** τ varie. L'**AUC** résume la capacité de **rang** : un positif tiré au hasard a-t-il un score plus élevé qu'un négatif ? L'AUC ne remplace pas l'analyse des coûts. Deux modèles de AUC proche peuvent différer fortement sur les FN coûteux.

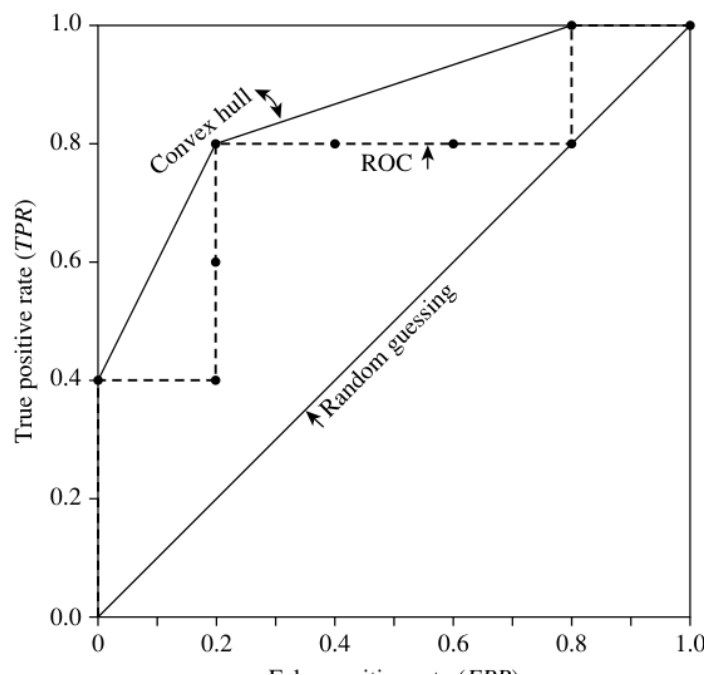


FIGURE 2 – Courbe ROC construite à partir de scores probabilistes et d'un balayage de seuils. (Source : Han, Kamber et Pei, *Data Mining: Concepts and Techniques*, 3e éd., Figure 8.19.)

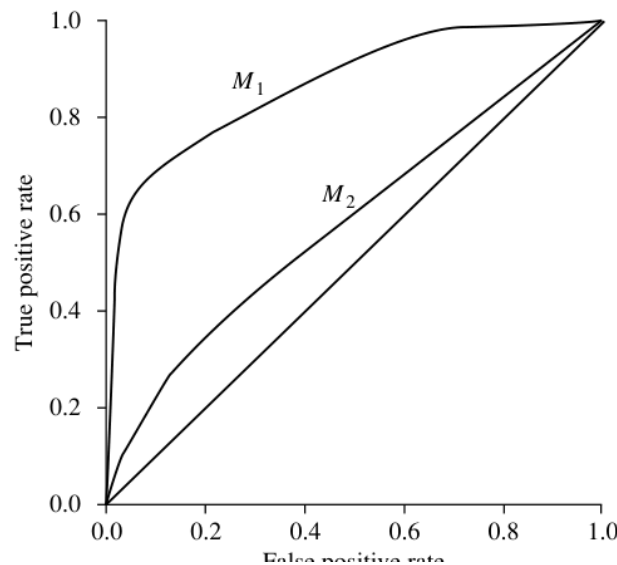


FIGURE 3 – Comparaison de deux modèles via leurs courbes ROC (diagonale = tirage aléatoire). (Source : Han, Kamber et Pei, *Data Mining: Concepts and Techniques*, 3e éd., Figure 8.20.)

9 Validation croisée et pipelines

Plusieurs stratégies de découpage existent :

- **Holdout.** Couper une fois en train / test (simple, mais score instable si n petit) ;
- **Sous-échantillonnage aléatoire répété.** Plusieurs découpes, moyenne des scores ;
- **Validation croisée k -fold.** k plis ; chaque pli sert une fois de test ;
- **Bootstrap** (option avancée). rééchantillonner avec remise pour estimer la variance.

La **stratification** garde la proportion de classes dans chaque pli (important si défaut rare). Toute transformation apprise (moyenne d'imputation, paramètres du scaler) doit être dans un **Pipeline** ré-entraîné à chaque fold. Comparez les modèles sur le **même protocole** ; en recherche, on peut tester si l'écart de score est statistiquement significatif.

Exemple commenté

Iris en binaire (setosa vs autres). Pour k -NN, $k = 1$ montre une variance de score CV élevée ; $k = 5$ stabilise l'exactitude. La régression logistique avec **StandardScaler** atteint souvent des performances similaires avec une frontière linéaire plus stable. Comparer les deux familles avant d'investir dans des modèles plus lourds.

Cas pratique

Contexte. Une banque veut repérer à l'avance les clients qui risquent de **ne pas rembourser** leur crédit. On dispose de dossiers *passés* (données simulées pour le TD) : pour chaque client, on connaît deux informations financières et si un **défaut** a eu lieu ou non.

Les colonnes de `credit_sim.csv`.

- **income** : revenu annuel (nombre). Plus le revenu est élevé, le risque de défaut tend à baisser (dans la simulation).
- **debt_ratio** : part de la dette par rapport au revenu (entre 0 et 1). Plus ce ratio est élevé, le risque de défaut tend à augmenter.
- **default** : la **cible** à prédire. Valeur 0 = pas de défaut, 1 = défaut observé.

Ce que vous devez faire (exercice 2.4 du TD).

1. Générer le fichier en exécutant `exercices/correction_ex24_credit.py` (ou recopier la même logique dans votre notebook).
2. Regarder le taux de défaut et des statistiques simples (moyenne, min, max) sur **income** et **debt_ratio**.
3. Couper les données en **train** (80 %) et **test** (20 %), en **stratifiant** sur **default** pour garder la même proportion de défauts dans chaque partie.
4. Entraîner sur le train uniquement un **pipeline** : d'abord **StandardScaler** (mise à l'échelle), puis **LogisticRegression**. Le scaler apprend les moyennes et écarts-types *sur le train* ; le test ne sert qu'à l'évaluation finale.
5. Mesurer l'**AUC-ROC** sur le **test** (pas sur le train). Objectif pédagogique : $AUC \geq 0,75$ (le modèle classe mieux qu'un tirage au hasard).
6. Lire les **coefficients** de la régression logistique *après* standardisation : signe (+ augmente le risque, – le diminue) et ordre de grandeur (quelle variable pèse le plus ?).

Comment interpréter sans vous perdre. L'AUC résume « est-ce que les clients qui ont fait défaut ont en moyenne un score plus élevé que les autres ? ». Les coefficients disent « si toutes les autres variables restent fixes, que se passe-t-il quand **debt_ratio** ou **income** augmente d'un écart-type ? ». Ce n'est pas encore la décision « accorder ou refuser » : le seuil et les coûts viennent au chapitre 4.

Fichiers utiles. Script complet de référence : `exercices/correction_ex24_credit.py`. Pour comparer k-NN, logistique et arbre avec matrice de confusion et courbe ROC sur un *autre* jeu (cancer du sein), voir `exemples/02_classification_metrics.ipynb`.

10 Classes déséquilibrées

Lorsque la classe positive représente moins de 5 % des cas (fraude, défaut rare), l'exactitude devient une métrique **trompeuse**. Plusieurs stratégies existent. **ponds de classe** (`class_weight='balanced'`), sur- ou sous-échantillonnage sur le train uniquement, métriques adaptées (AUC-PR, rappel à précision fixée). Le sur-échantillonnage du test fausserait l'évaluation : le déséquilibre du test reflète la réalité opérationnelle. Le choix dépend du coût métier. En anti-fraude, un rappel élevé peut primer ; en marketing, une précision élevée limite le spam. Ces préférences se formalisent au chapitre 4 avant de figer τ .

11 Courbe précision-rappel

La courbe ROC peut être optimiste sous fort déséquilibre. La courbe **précision-rappel** trace la précision en fonction du rappel quand τ varie ; l'AUC-PR pénalise les modèles qui produisent beaucoup de faux positifs. Comparer deux modèles uniquement sur ROC sans examiner PR est une erreur fréquente en projet étudiant.

12 Ensembles : bagging et boosting

L'idée est de combiner plusieurs modèles « faibles » pour un modèle plus stable.

- **Bagging**. Entraîner plusieurs arbres sur des sous-échantillons tirés avec remise, puis voter (forêt aléatoire) ;
- **Boosting** (AdaBoost). entraîner des modèles en série en **repondérant** les erreurs précédentes ;
- **Forêt aléatoire**. Bagging + tirage aléatoire de variables à chaque split.

Pour les **classes déséquilibrées**, on peut recourir à rééchantillonnage, métriques adaptées, poids de classe. En production, une logistique ou un arbre peu profond reste souvent plus simple à expliquer qu'une forêt.

13 Régularisation et sur-apprentissage en logistique

La régression logistique peut être étendue avec une pénalité L2 sur $\|\mathbf{w}\|^2$ (*ridge*), réduisant l'amplitude des coefficients et stabilisant l'estimation lorsque p est grand ou colinéaire. La pénalité L1 (*lasso*) pousse certains β_j exactement à zéro, effectuant une sélection de variables. Le paramètre de régularisation C (inverse de la force de pénalité en scikit-learn) se choisit par grille sur validation croisée. Trop petit, underfitting ; trop grand, coefficients explosent. Interpréter les coefficients exige de rappeler qu'ils décrivent l'effet **conditionnel** des autres variables fixées. Une variable corrélée à une autre voit son coefficient dilué ou instable ; d'où l'intérêt de la régularisation et de l'analyse VIF en amont (chapitre 1). La régularisation pénalise de grands coefficients (L2 ridge, L1 lasso) pour limiter le sur-apprentissage ; on parle de compromis **biais-variance** entre modèle trop simple et trop complexe.

14 Questions de révision guidées

1. Pourquoi la standardisation est-elle quasi indispensable avant k-NN sur des variables hétérogènes ?
2. Donnez un exemple métier où maximiser le rappel est préférable à maximiser la précision.
3. Expliquez pourquoi l'exactitude peut être élevée alors que le modèle ne détecte aucun positif.
4. Quelle est la différence entre optimiser l'AUC et optimiser le coût espéré ?
5. Pourquoi doit-on inclure le `StandardScaler` dans le pipeline de validation croisée ?
6. Comparez, en une phrase chacun, biais et variance dans le cas d'un arbre non élagué.

7. Quel rôle joue la stratification dans une CV sur un jeu à 2 % de positifs ?
8. Citez deux limites du Naïve Bayes et une situation où il reste pertinent.

15 Erreurs fréquentes en évaluation

Mélanger train et test avant le prétraitement ; optimiser sur le test ; choisir le modèle qui maximise le hasard sur un petit test ; ignorer la dimension temporelle ; comparer des modèles entraînés sur des features différentes ; confondre AUC élevée et probabilités bien calibrées. Chaque point correspond à un biais mesurable qui gonfle artificiellement les slides de fin de projet.

16 Synthèse méthodologique

1. Définir la cible, l'horizon et interdire toute variable du futur.
2. Séparer train / validation / test (ou CV) avant tout `fit`.
3. Comparer plusieurs modèles avec la **même** métrique alignée sur le métier.
4. Documenter FN et FP. Qui paye le coût de l'erreur ?
5. Fournir scores, matrice de confusion et courbe ROC au comité ; le seuil sera fixé au chapitre 4.

17 Lecture des odds et rapport d'évaluation

Rapport $\frac{p}{1-p}$ où p est la probabilité de la classe positive. Odds = 2 signifie « deux fois plus de chances oui que non ». En logistique, une unité de plus sur une variable (standardisée) multiplie les odds par e^{β_j} ; si $\beta_j = 0,7$, les odds sont multipliés par environ 2. En régression logistique, ce langage est courant en épidémiologie et en crédit pour lire l'effet d'une variable sur le risque. Un rapport d'évaluation Master 2 minimal contient une description de $\mathcal{D}$, protocole de split, tableau comparatif des modèles (AUC, F_1 , temps CPU), matrice de confusion au seuil retenu provisoirement, courbes ROC et PR superposées, liste des variables les plus influentes, limites connues (variables manquantes structurelles, période COVID, etc.). L'objectif n'est pas d'« atteindre 100 % » mais de montrer un raisonnement critique. Pourquoi ce modèle plutôt qu'un autre, où il se trompe, ce qu'il faudrait mesurer de plus si le temps projet le permettait.